CSTAM 3.0 – TuniBot Challenge

TuniBot

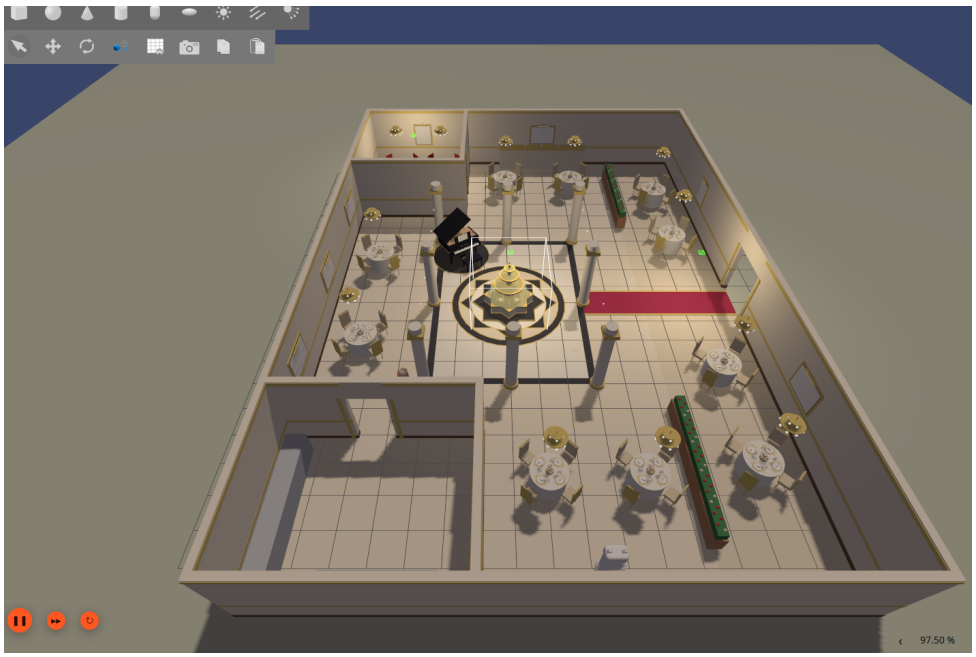
Autonomous Service Robot for Indoor Delivery

Technical Report – Phases 1, 2 and 3

Team ConnectENIT

Wassim Toukebri Maram Hammami Ridha Charrad

IEEE ENIT Computer Society Student Branch Chapter



November 1, 2026

Repository: https://github.com/maramhammami-svg/tunibot-phase3

Contents

Abstract **2**

1 Team Overview and Contributions **2**

2 Problem Statement and Industrial Context **2**

3 Proposed Solution **3**
3.1 Requirements coverage 4

4 System Design **4**
4.1 Architecture 4
4.2 Simulation environment 6
4.3 Mapping and localization 6
4.4 Navigation 7
4.5 Delivery task manager 7
4.6 Auto-docking and battery 9
4.7 Dynamic obstacle avoidance 9
4.8 Web interface (Phase 3) 10
4.9 ROS 2 interface 11

5 Mockups **11**
5.1 Mission Control simulator (Phases 1 and 2) 11
5.2 Design direction (Phase 3) 12
5.3 Client page 13
5.4 Admin control room 14

6 Bonus Functionalities **15**
6.1 Voice command integration for placing orders 15
6.2 Additional features beyond the requirements 17

7 Technologies, Frameworks and Tools **18**

8 Development Milestones **19**

9 Testing Results **19**
9.1 Automated tests 19
9.2 Phase 2 test cases 20
9.3 Scenario tests in Gazebo 20
9.4 Issues found and fixed 20

10 Future Improvements **21**

References **21**

Abstract

TuniBot is a complete ROS 2 software stack for an autonomous waiter robot that serves a restaurant from the guest's order to the robot's return to its charger, with no human in the loop. It runs on ROS 2 Jazzy, Gazebo Harmonic and Nav2, on a simulated BellaBot differential-drive robot with a 2D LiDAR, in two restaurant worlds: an open lounge with 24 tables and *Dar Tunibot*, a traditional Tunisian restaurant with 1.20 m arched doorways.

The project was built in three phases. **Phase 1** delivers the foundations: SLAM mapping, AMCL localization, Nav2 navigation to every table, a delivery task queue with editing, cancellation and preemption, and automatic return to the dock. **Phase 2** makes the robot robust: a walking pedestrian that the robot detects and plans around, costmaps tuned for narrow doorways with a multi-stage recovery, and a physical 24 V Li-ion battery model with a low and critical return-to-dock policy. **Phase 3** adds the user layer: a web Client page where guests pick their table on the floor plan and order by tapping or by voice (French, English and Tunisian Darija), and an Admin control room where staff watch the robot, its battery and its queue live. Voice command integration for placing orders is our bonus functionality. The web layer is a separate ROS 2 package that only uses existing topics.

The system is covered by 65 automated tests (9 Phase 1 unit tests, 5 Phase 2 integration tests and 51 web tests), all passing, and by end-to-end runs in Gazebo where a web order was delivered to its table.

1 Team Overview and Contributions

We are **ConnectENIT**, a team of three students from the IEEE ENIT Computer Society Student Branch Chapter. All three of us worked on every phase of the challenge: we designed, coded, tested and reviewed together. To move fast, each member led one area in each phase while the others reviewed and tested it, as summarised in Table 1.

Member	**Phase 1: Foundations**	**Phase 2: Integration**	**Phase 3: UI/UX & docs**
Wassim Toukebri	Nav2 navigation set-up, waypoints to tables	Dynamic obstacle avoidance, costmap tuning for narrow doorways, recovery behaviours	FastAPI server and ROS 2 bridge, order tracking, Gazebo validation
Maram Hammami	Delivery task queue manager and auto-docking	Battery model and return-to-dock policy, Dar Tunibot world	Client page UX and visual design, voice ordering (French, English, Darija), technical report
Ridha Charrad	Gazebo world and robot model, SLAM mapping, AMCL localization	Integration tests, pedestrian scenarios	Admin control room, automated tests, demo video

Table 1: Each member's lead area per phase. All members contributed to all three phases.

2 Problem Statement and Industrial Context

Restaurant service still depends on staff walking every plate and drink from the kitchen to the table. That repetitive work takes time away from guests, and it scales badly at peak hours. Service robots such as the BellaBot are now common in Asian restaurants, but they need software that copes with real venues: narrow aisles, people walking around, a battery that runs out and guests who want to know where their order is.

The CSTAM 3.0 challenge, set with our industrial partner TuniBot, a Tunisian robotics and automation company, asks for a software-only answer built in three phases. The robot must:

- map an indoor environment and localize itself in it;
- receive food and beverage requests and queue them;
- drive autonomously to predefined tables to deliver them, safely around furniture and people;
- return to its dock to charge when idle or low on battery;
- handle narrow passages and moving obstacles, and recover when blocked (Phase 2);
- give guests an interface to order and staff an interface to supervise the robot (Phase 3).

Two constraints shaped the design. Everything runs in simulation with industry-standard open-source tools (ROS 2, Gazebo, Nav2), so the stack can move to a real robot later. And the result must work in a realistic venue: besides a large lounge, we built Dar Tunibot, a traditional Tunisian restaurant whose 1.20 m arched doorways and narrow aisles are a hard case for any navigation stack.

3 Proposed Solution

Our solution is a layered ROS 2 system: a simulated restaurant and robot at the bottom, the robot's core logic in the middle and a web interface on top. Each layer talks to the next only through ROS 2 topics and actions, so each can be tested and replaced on its own.

Layer	**What it does**	**Phase**
Simulation	Gazebo Harmonic worlds (Restaurant Lounge, 24 tables; Dar Tunibot, 10 tables), BellaBot differential-drive model with 2D LiDAR and wheel odometry, a walking pedestrian	1, 2
Mapping and localization	SLAM Toolbox to build the maps, AMCL to localize, auto-initialized at the dock	1
Navigation	Nav2: NavFn global planner, DWB local controller, behaviour tree and recoveries; costmaps tuned for 1.20 m doorways	1, 2
Delivery task manager	FIFO queue of delivery requests, dispatch to tables, modify and delete, preemption, recovery that keeps the order queued	1, 2
Auto-docking	Return to dock when idle, docking sequence, rest and charge	1
Dynamic obstacles	Pedestrian controller (patrol, cross, block, clear); 10 Hz local costmap so Nav2 re-plans around people	2
Battery	24 V 20 Ah Li-ion NMC model with voltage sag and temperature; return to dock at 25 %, emergency dock at 15 %, recharge to 80 %	2
Operator tools	Mission Control simulator (Tkinter GUI and CLI) for dispatching and testing	1, 2
Web interface	Client ordering page with voice orders, Admin control room, FastAPI bridge to ROS 2	3

Table 2: Layers of the TuniBot software stack.

3.1 Requirements coverage

Phase	Requirement	How we met it
1	Simulated environment	Two Gazebo worlds with tables, kitchen and dock; BellaBot model with LiDAR and odometry
1	Mapping and localization	SLAM Toolbox maps of both worlds; AMCL localization initialized at the dock
1	Autonomous navigation	Nav2 goals to 24 + 10 predefined table waypoints, kitchen and dock
1	Delivery task management	FIFO queue with live edit, delete, cancel and preemption of docking by a new order
1	Auto-docking	Return to dock after an idle timeout or on command; stop docking at any time
2	Dynamic obstacle avoidance	Pedestrian in four modes; fast local costmap so Nav2 re-plans around the person
2	Complex layout handling	Inflation tuned for 1.20 m doorways; reverse, clear and re-plan recovery, up to 3 retries
2	Battery and state simulation	Physical Li-ion model; 25 % / 15 % / 80 % docking policy
3	Ordering interface (web or mobile)	Responsive Client page: table on the floor plan, menu book, voice ordering, live tracking, several orders
3	Admin view: robot status and battery	Admin control room: live map with Nav2 path, mission, battery, queue, health, log, controls
3	Documentation	This report, the repository README and the demonstration video

Table 3: Challenge requirements and how they are met.

4 System Design

4.1 Architecture

Figure 1 shows the whole system. The Phase 1 and Phase 2 nodes form the robot; the Phase 3 web package sits on top and reaches the robot only through the topics the other nodes already use.

The workspace has four ROS 2 packages:

- `cstam_gazebo`: worlds, BellaBot and pedestrian models, Gazebo–ROS bridge (Phases 1–2);
- `cstam_navigation`: Nav2, AMCL and SLAM parameters, maps, table waypoints, launch files (Phases 1–2);
- `cstam_core`: task manager, docking controller, battery simulator, pedestrian controller (Phases 1–2);
- `cstam_web`: web server, ROS 2 bridge and the Client and Admin pages (Phase 3).

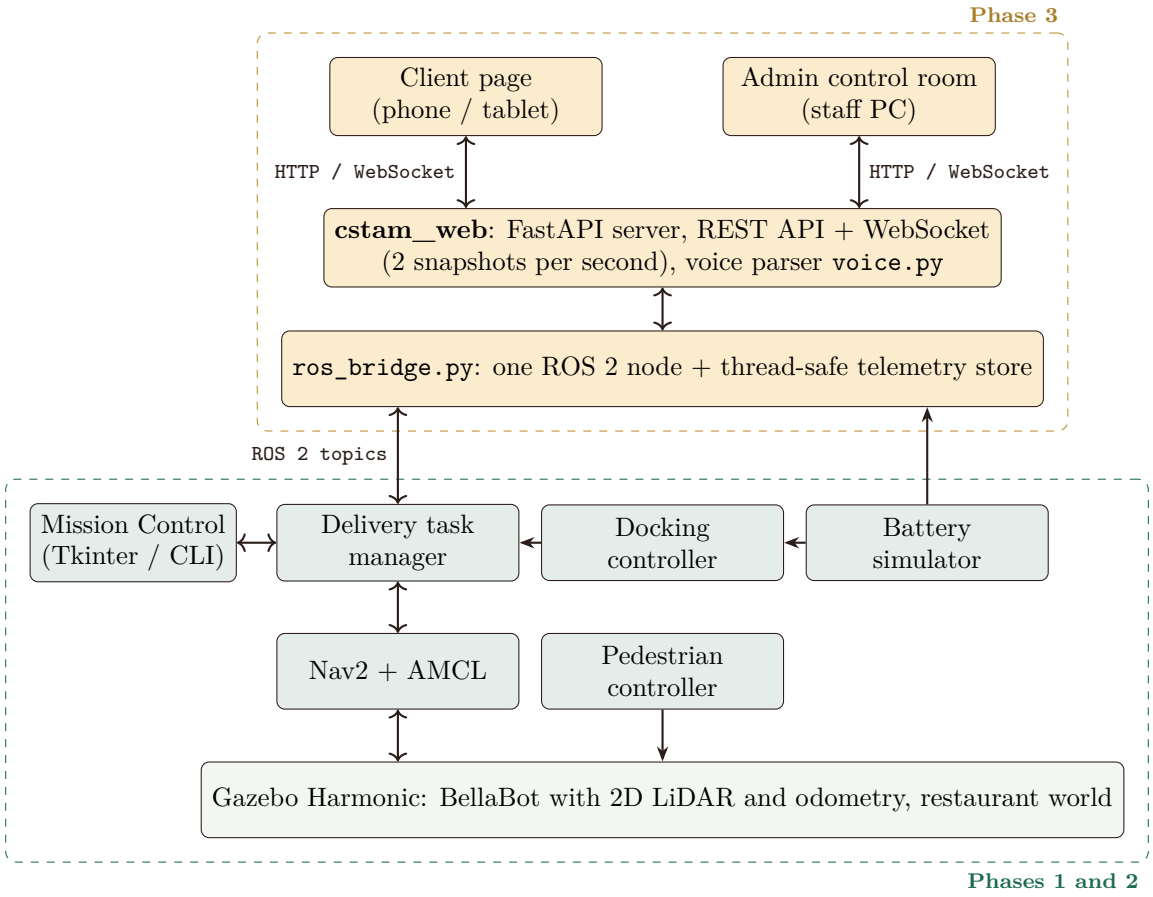


Figure 1: System architecture. Arrows are ROS 2 topics or actions, except HTTP / WebSocket between the browsers and the web server.

4.2 Simulation environment

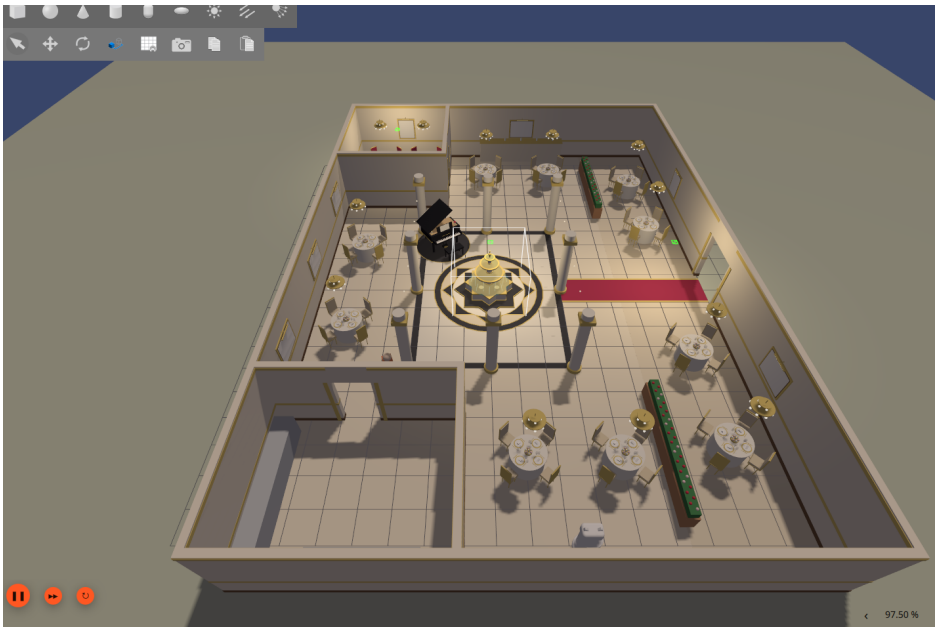
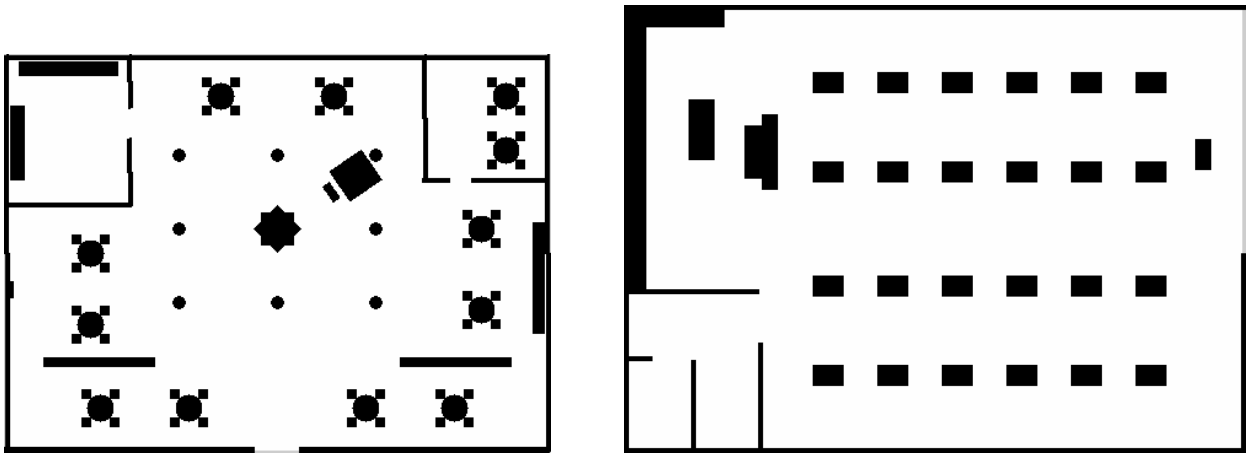


Figure 2: The Restaurant Lounge world in Gazebo Harmonic (24 tables, kitchen counter and dock).

- **Restaurant Lounge** (`restaurant.world`): 24 tables in four rows, a kitchen pickup counter on the west side and a resting dock in the south-east corner at $(7.06, -12.00)$.
- **Dar Tunibot** (`resto_arbi.world`): a traditional Tunisian house with a central fountain, a north-wing kitchen, 10 tables, a dock on the west wall and 1.20 m stone arch doorways.
- **BellaBot**: a differential-drive base (radius 0.28 m, top speed 0.45 m/s) with a 2D LiDAR at $z \approx 0.2$ m and wheel odometry, bridged to ROS 2 (`/scan`, `/odom`, `/cmd_vel`).
- **Pedestrian**: a 1.70 m human model on a hidden differential-drive chassis, so it stays upright and is seen by the LiDAR (see Section 9.4).

4.3 Mapping and localization

We built one occupancy-grid map per world with SLAM Toolbox, driving the robot through the restaurant (`run_mapping_demo.sh`) and saving the result with `save_map.sh` (Figure 3). At run time the robot localizes on that map with AMCL. Its initial pose is published automatically at the dock, so the robot is localized as soon as the system starts, with no manual "2D Pose Estimate".



(a) Dar Tunibot. (b) Restaurant Lounge.

Figure 3: Occupancy-grid maps built with SLAM Toolbox.

4.4 Navigation

Navigation uses Nav2 with the NavFn global planner, the DWB local controller and the default navigate-to-pose behaviour tree with its recoveries. Each table has a waypoint in a YAML file (`waypoints.yaml`, `resto_arbi_waypoints.yaml`): a service point beside the table, facing it, so the robot never drives into chairs. The task manager sends these poses through the `navigate_to_pose` action.

Narrow doorways. With default settings, Nav2's inflation layer blocks Dar Tunibot's 1.20 m doors, because the inflated zones of the two door posts overlap. We kept the real robot radius and tuned the inflation instead:

$$r_{\text{robot}} = 0.28\,\text{m}, \qquad r_{\text{inflation}} = 0.42\,\text{m (from 0.55)}, \qquad k_{\text{scale}} = 4.5$$

$$W_{\text{free}} = W_{\text{door}} - 2\,r_{\text{robot}} = 1.20 - 0.56 = 0.64\,\text{m}.$$

The steeper cost decay keeps the robot on the centreline of the doorway while leaving a free corridor for the planner.

4.5 Delivery task manager

The task manager (`delivery_task_manager.py`) is the robot's brain. It receives orders on `/delivery_request` (`{"target": "Table 4", "item": "2x Thé à la menthe"}`), gives each one an id (`TASK-0001`, ...) and keeps a FIFO queue. When the robot is free, it pops the next task and sends the Nav2 goal. On arrival it waits at the table for the hand-over, marks the task completed and pops the next one. Its state machine is shown in Figure 5.

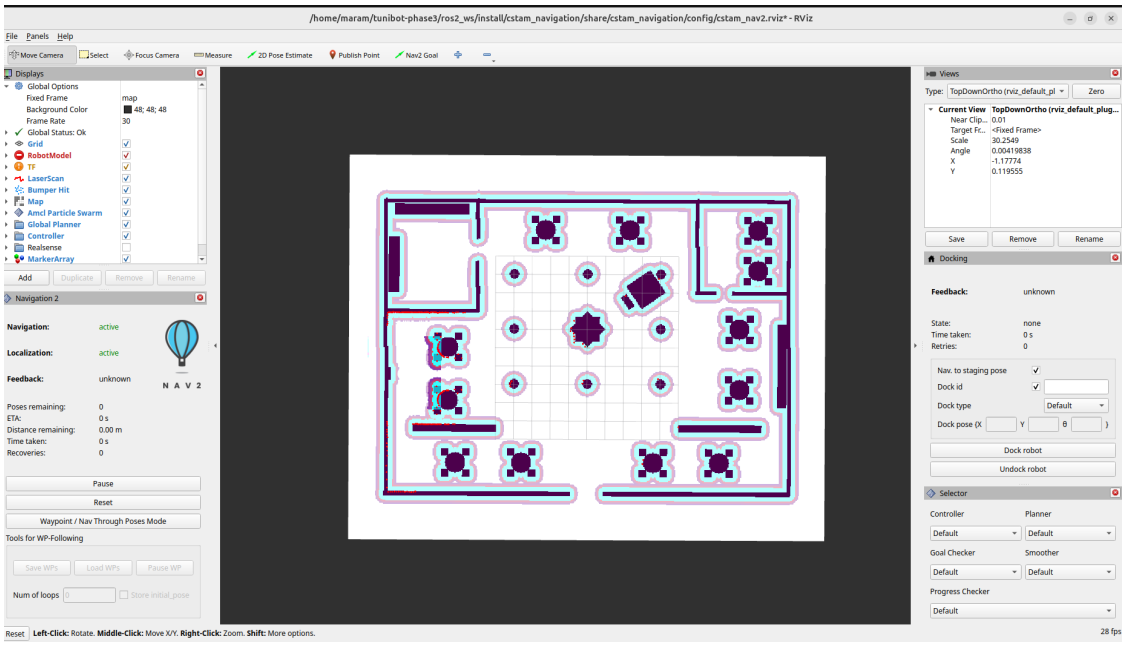


Figure 4: Nav2 in RViz (Restaurant Lounge): the SLAM map with the global and local costmaps; the inflated zones around tables and walls keep the robot at a safe distance.

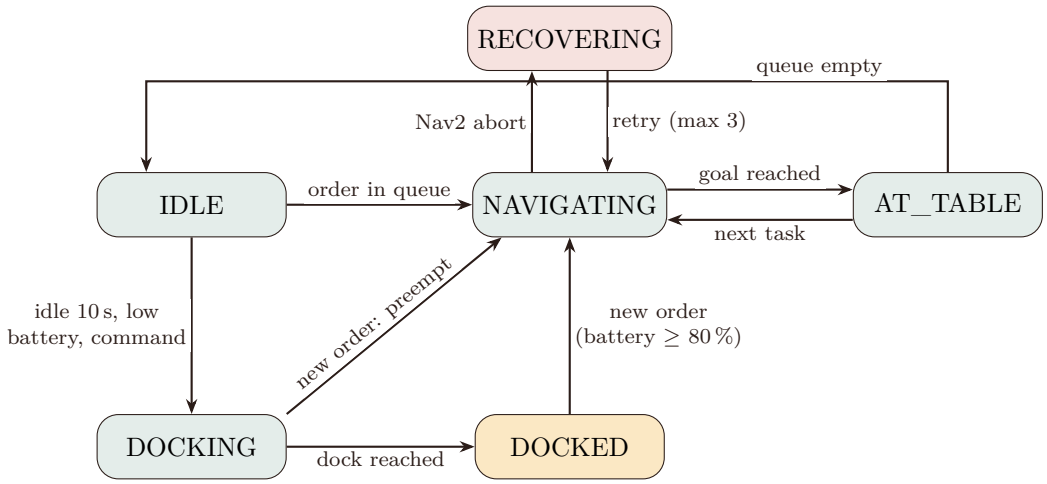


Figure 5: Simplified state machine of the delivery task manager.

- **Live queue editing:** a pending task can be modified (table or item) or deleted through `/delivery_task_action`, without stopping the robot. An active task can be cancelled.
- **Preemption:** a new order that arrives while the robot is docking cancels the docking goal and is served at once (unless the battery is low).
- **Recovery:** when Nav2 aborts, the manager stops the robot, clears the costmaps, backs up ($v_x = -0.15\,\mathrm{m/s}$ for $1.8\,\mathrm{s}$) and re-plans, up to 3 times. If the table is still unreachable, the order goes back into the queue and the operator is alerted, so no order is lost.
- **Status:** the manager publishes `/robot_system_state` and a JSON `/delivery_queue_status` (current task, pending tasks, completed count, battery lock), which every interface reads.

4.6 Auto-docking and battery

Battery model. `battery_simulator.py` models a 24 V 20 Ah Li-ion NMC pack (7S, 480 Wh). The open-circuit voltage follows a non-linear curve between $V_{\min} = 21.0\,\mathrm{V}$ and $V_{\max} = 29.4\,\mathrm{V}$:

$$V_{\mathrm{ocv}}(s) = V_{\min} + (V_{\max} - V_{\min})\left[0.84\,s + 0.16\,s^{0.3} - 0.05\,(1-s)^6\right]$$

where s is the state of charge. The load depends on motion, $P = 35 + 65\,|v| + 30\,|\omega|\,\mathrm{W}$ (35 W idle, about 130 W when cruising), and the terminal voltage sags under load through the internal resistance, $V = V_{\mathrm{ocv}} - I\,R_{\mathrm{int}}$ with $R_{\mathrm{int}} = 0.045\,\Omega$. Charge is counted in coulombs; charging follows a CC–CV profile at 6 A, and pack temperature is tracked. The node publishes the standard `/battery_state` and an extended JSON `/battery_status` at 1 Hz.

Docking policy. `docking_controller.py` checks the battery twice a second and decides when to go home:

State	Battery	Action
Normal	$> 25\,\%$	Return to dock after 10 s idle with an empty queue; a new order preempts it
Low	$\leq 25\,\%$	Finish the current order, then `DOCK_NOW:dock_low_battery`; no preemption
Critical	$\leq 15\,\%$	Emergency `DOCK_NOW:dock_critical_battery`, even during a delivery; the order stays queued
Charging	$< 80\,\%$	Robot stays docked; new orders wait until 80 %

Table 4: Return-to-dock policy.



Figure 6: The robot parked and charging at its dock.

4.7 Dynamic obstacle avoidance

A pedestrian model walks through the restaurant, driven by `dynamic_obstacle_controller.py` in four modes: PATROL (walks up and down an aisle at 0.35 m/s), CROSS (crosses the robot's path), BLOCK (steps into the path and stands still) and CLEAR (steps aside). The LiDAR sees the person, and the local costmap, updated at 10 Hz with 5 m ray-tracing, marks the person and clears the cells behind as they move. Nav2's controller and planner then steer the robot around the person, or re-plan another route when the aisle is blocked, and the delivery continues. The controller

publishes the pedestrian's position and mode on `/dynamic_obstacle/status` and RViz markers on `/dynamic_obstacle/markers`.

4.8 Web interface (Phase 3)

- `ros_bridge.py`: one ROS 2 node that subscribes to the robot topics and publishes the web commands.
- `store.py`: a thread-safe telemetry store that mirrors what the robot publishes and derives each web order's status.
- `server.py`: FastAPI REST endpoints, plus a WebSocket that pushes a full snapshot twice a second.
- `voice.py`: the voice-order parser (plain Python, no ROS).
- `static/`: Client and Admin pages in plain HTML, CSS and JavaScript, with no build step. Dish illustrations and icons are inline SVG, and the floor map is drawn on a canvas from the world file.
- `mock.py`: a simulated robot that reuses the real task queue and battery classes, for demos without Gazebo (`--mock`).

Order tracking without changing the task manager. The task manager knows nothing about web orders. The bridge links an order to its task by watching the queue status: the first new task id for the same table with the same item text is that order. From then on its status follows the task: queued, en route, at the table. When it leaves the queue, a rise in the completed count means delivered; otherwise it is reported as failed or cancelled. An order not picked up within 8 s is shown as not accepted. Each order is tracked on its own, so a guest can follow several at once, and they are kept in the browser so tracking resumes after a reload.

Voice ordering. The client page can also take a spoken order; this bonus feature is described in Section 6.

4.9 ROS 2 interface

Topic / action	From → to	Content
`/delivery_request`	UI → task manager	new order (JSON: target, item)
`/delivery_task_action`	UI → task manager	modify, delete or cancel a task (JSON)
`/dock_command`	controller / UI → task manager	`DOCK_NOW:<reason>` or stop docking
`/dock_state`	task manager → all	docked or not (Bool)
`/delivery_queue_status`	task manager → all	current task, pending tasks, completed count, battery lock (JSON)
`/robot_system_state`	task manager → all	IDLE, NAVIGATING, AT_TABLE, RECOVERING, DOCKING, DOCKED
`navigate_to_pose`	task manager → Nav2	action: goal pose of a table, kitchen or dock
`/cmd_vel`, `/scan`, `/odom`	Nav2 ↔ Gazebo	velocity commands, LiDAR, odometry
`/amcl_pose`, `/plan`	Nav2 → UI	robot pose and global path
`/battery_state`, `/battery_status`	battery → controller / UI	percentage, voltage, current, power, temperature, flags (1 Hz)
`/dynamic_obstacle/cmd_vel`, `/odom`	controller ↔ Gazebo	drive the pedestrian, read its pose
`/dynamic_obstacle/status`	controller → UI	pedestrian position and mode (JSON)
`/delivery_waypoints`	web bridge → UI	world name and table coordinates (latched)

Table 5: Main ROS 2 topics and actions.

5 Mockups

The screens below are the working interfaces, captured from the running system. They are the final UI, not drawings.

5.1 Mission Control simulator (Phases 1 and 2)

During Phases 1 and 2, we drove and tested the robot from a Mission Control simulator written in Tkinter (`delivery_simulator_ui.py`, also available as a command-line tool for SSH sessions). It dispatches orders with menu presets, shows the live queue with edit and delete, controls docking, sets the battery to low or critical, and switches the pedestrian's mode.

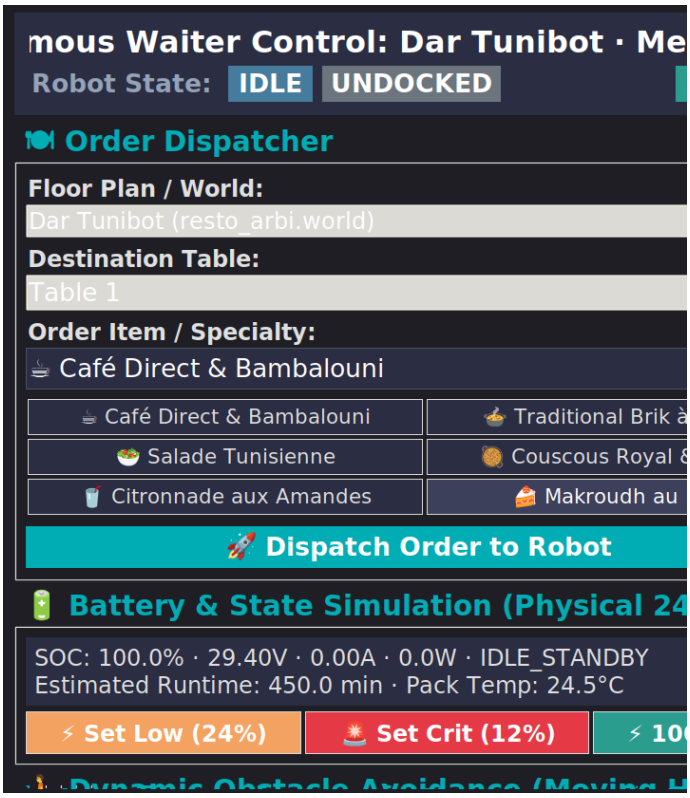


Figure 7: Mission Control simulator: order dispatcher and battery panel (Dar Tunibot).

5.2 Design direction (Phase 3)

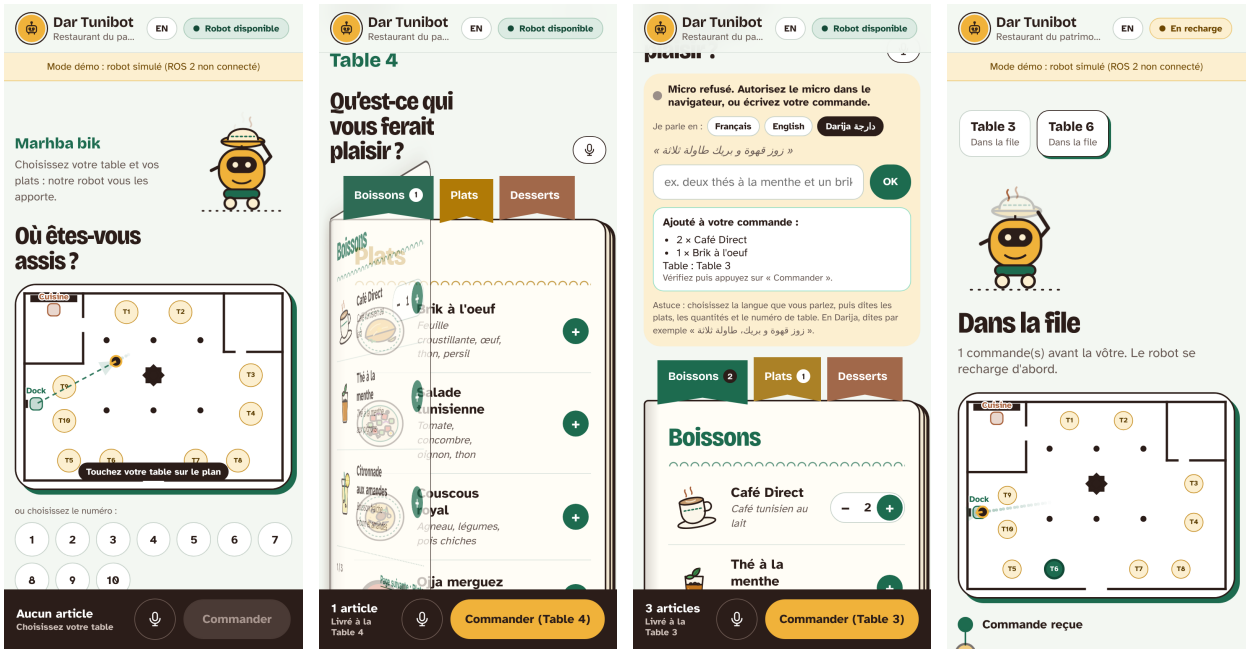
For guests and staff we wanted an interface that feels like it belongs in a Tunisian restaurant rather than a generic dashboard. The visual identity borrows from Nabeul pottery: copper green, ochre and manganese brown on a pale slip background. Headings use Bricolage Grotesque and body text uses Atkinson Hyperlegible, a typeface designed for readability. Each dish has its own flat illustration (Figure 8), and the menu behaves like a real menu book with page turns. Motion is used only where it carries meaning (a dish flying to the tray, the robot driving on the map) and is turned off for users who ask for reduced motion.



Figure 8: Dish illustrations drawn for the Tunisian menu.

5.3 Client page

The guest orders in three taps: table, dishes, Order. Everything else on the page answers the guest's next question: is the robot free, where is my order, when will it arrive.



(a) Pick the table on the floor plan. (b) Menu book, one page per category. (c) Voice order in Darija. (d) Several orders tracked at once.

Figure 9: Client page on a phone.

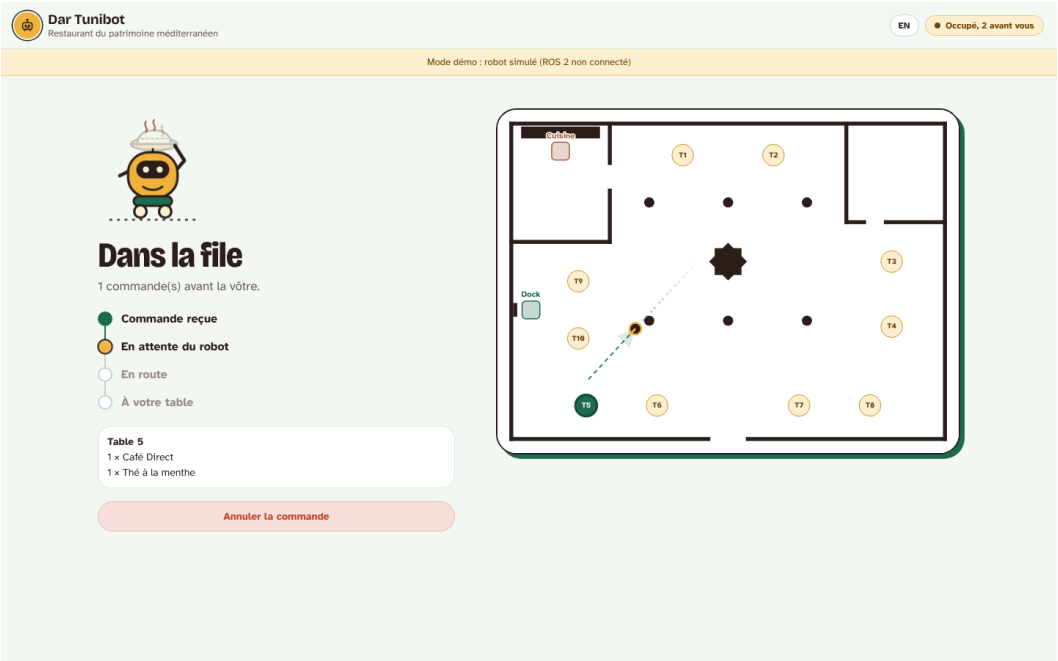


Figure 10: Client page on a wide screen: the floor plan stays visible beside the order and the live tracking.

Edge cases handled.

- Robot offline (no battery heartbeat for 3 s): red banner, ordering disabled.

- Battery lock (robot recharging to 80 %): order accepted; the guest is told it leaves after charging.
- Order not picked up by the task manager within 8 s: “not accepted, please ask the staff”.
- Navigation failure: “a waiter will bring your order” instead of a silent stop.
- Page reloaded or phone locked: orders are kept and tracking resumes.
- Lost Wi-Fi: the live connection reconnects on its own.

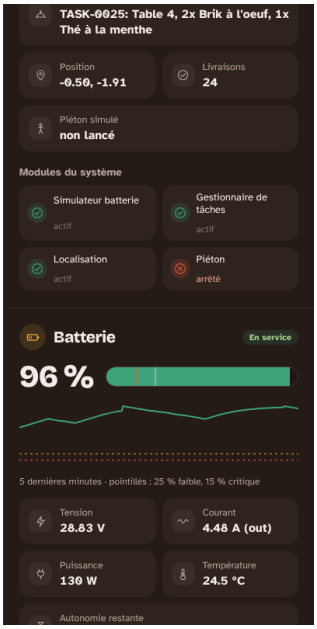
French is the default and English is one tap away; voice recognition has its own language choice, because recognition must follow the language the guest speaks, not the page language.

5.4 Admin control room

The Admin view at `/admin` shows, on one screen, everything staff need to trust the robot: where it is, what it is doing, how much battery it has left and what is waiting. The live floor map takes most of the screen, a mission card floats over it, and the side column keeps every panel open: orders, robot, battery, log and demo mode.



(a) Live map with the Nav2 path, mission card and order tickets.



(b) Robot and battery panels.

Figure 11: Admin control room.

Panel	Shows	Actions
Floor map	Walls, tables, dock and kitchen from the world file; robot pose, trail and Nav2 path; pedestrian; pending orders per table	Tap a table to send the robot there
Mission card	Current mission, state, battery ring, deliveries completed	–
Orders	Current and pending orders as kitchen tickets	Cancel any order
Robot	Task, position, deliveries, pedestrian, health of each module	Return to dock, cancel the return, send to kitchen or a table
Battery	Percentage, voltage, current, power, temperature, runtime, 5-minute history	–
Log	Last events: orders, state changes, battery warnings, voice orders	–
Demo mode	–	Battery 100 %, 24 % or 14 %; pedestrian patrol, cross, block or absent

Table 6: Admin panels.

6 Bonus Functionalities

The bonus functionality we implemented is **voice command integration for placing orders**. A guest can order by speaking, in French, English or Tunisian Darija, instead of tapping through the menu. We also added several extras beyond the required interface; they are listed in Section 6.2.

6.1 Voice command integration for placing orders

What the guest does. On the Client page the guest taps the microphone button, picks the spoken language (Français, English or Darija) and says the whole order in one sentence, for example:

- French: *"deux thés à la menthe et un brik pour la table 4"*
- English: *"two coffees and a couscous, table seven"*
- Darija: *"zouz kahwa w brik, tawla tletha"* (two coffees and a brik, table three), also understood when the browser returns it in Arabic script.

The cart and the table fill in, the menu book opens on the page of the first dish, and the page reads the order back aloud. The guest then confirms with the Order button: nothing is sent to the robot before that tap, so a misheard word can never send the robot to the wrong table.

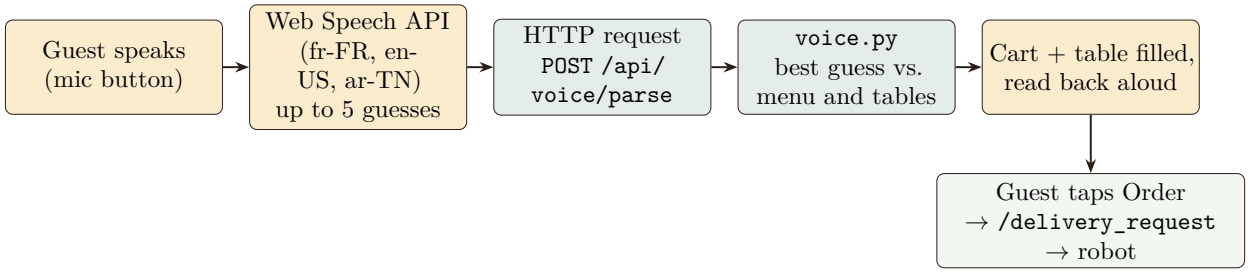


Figure 12: Voice ordering pipeline: the browser does speech-to-text, our parser turns the text into an order, the guest confirms.

How it works.

1. **Speech to text, in the browser.** Chrome or Edge turns the speech into text with the Web Speech API, in the language chosen in the voice panel (`fr-FR`, `en-US` or `ar-TN` for Darija).

Listening continues through short pauses and stops after 2.5 s of silence. The browser is asked for up to five alternative transcripts.

2. **Parsing, on our server.** All alternatives go to `POST /api/voice/parse`. The parser `voice.py` (plain Python, no ROS, works offline) reads each one against the menu and the tables of the world that is running, and keeps the alternative that fits best: most dishes recognised, a table found, fewest unknown words.
3. **Confirmation.** The page fills the cart and the table, shows what it understood (and any word it could not place), and speaks the order back with speech synthesis. The guest confirms or corrects it by hand.

Element	Examples
Quantities	French *un, deux, trois…*; English *one, two, three…*; Darija *wahed, zouz, tletha, arba…*; digits, including Arabic-Indic digits
Tables	*table 4*, *table numéro 10*, *table seven*, *tawla tletha*, Arabic *taawila* with a number
Dish names and aliases	*thé à la menthe / atay / tay*, *café / kahwa*, *couscous / kousksi*
Speech-to-text mistakes	*brique* → brik, *bamba louni* → bambalouni, *makrout* → makroudh
Darija in Arabic script	known words mapped directly (numbers, *table*, dish names, "and"), the rest transliterated to Latin letters before matching
Filler words	*je voudrais*, *s'il vous plaît*, *please*, *n7eb*, *aatini* are ignored

Table 7: Language coverage of the voice parser.

What the parser understands. Matching combines accent folding, plural removal, a phonetic key and fuzzy string matching. A dish that is not on the menu is reported to the guest, never replaced by a guess, and a table that does not exist in the current world is rejected. Quantities are capped at nine per dish, like on the touch menu.

Design decision from testing. The first version used the page language for recognition. On an English page, French speech came back as nonsense ("…uventies and a birthday before"). The voice panel therefore has its own language choice, and French is the default because it is what most guests in Tunisia speak to a machine.

Testing. 28 automated tests cover the parser: French, English and Darija sentences in Latin and Arabic script, sound-alike spellings, unknown dishes, tables that do not exist, the quantity cap and the choice of the best alternative. Two browser end-to-end tests (Playwright) check the full flow: a voice order is parsed, confirmed, sent to the simulated robot and reaches "Delivered", and the microphone flow keeps the best of several alternatives. Figure 9 (c) shows a Darija order on a phone.

Limits. Speech recognition depends on the browser (Chrome or Edge) and on its online service, and browsers only allow the microphone on `localhost` or HTTPS. An on-device model is listed in the future improvements.

6.2 Additional features beyond the requirements

Feature	What it adds
Several orders at once	A guest can place a new order while earlier ones are still on their way and follow each one separately; orders survive a page reload
Table picked on the floor plan	The guest taps their table on the real restaurant plan, and can watch the robot drive to it
Arrival estimate	Remaining Nav2 path length turned into a time estimate while the robot drives
Bilingual interface	Client and Admin pages in French and English, with translated dish descriptions
Illustrated menu book	Menu with page turns and a drawing for every dish, following the world that is running (Tunisian menu for Dar Tunibot)
Battery monitoring panel	Live percentage, voltage, current, power, temperature, runtime and a 5-minute history with the 25 % and 15 % thresholds
Live control room	Robot, Nav2 path, pedestrian and pending orders per table on one map; tap a table to send the robot
Demo mode	Buttons to set the battery to low or critical and to change the pedestrian's behaviour, to show the robot's reactions live
Mock robot	The whole interface runs without ROS or Gazebo (`--mock`), reusing the real queue and battery classes, for demos and tests

Table 8: Additional features.

7 Technologies, Frameworks and Tools

Area	Technology	Use
Middleware	ROS 2 Jazzy	Nodes, topics, actions, launch files, colcon build
Simulation	Gazebo Harmonic, ros_gz	Restaurant worlds, BellaBot, LiDAR, pedestrian, bridge to ROS 2
Navigation	Nav2 (NavFn, DWB)	Path planning, control, behaviour tree, recoveries
Mapping / localization	SLAM Toolbox, AMCL	Building the maps, localizing on them
Visualisation	RViz 2	Map, costmaps, paths and markers
Robot logic	Python 3.12 (rclpy)	Task manager, docking, battery, pedestrian controller
Operator tool	Tkinter	Mission Control simulator (GUI and CLI)
Web server	FastAPI, Uvicorn, Pydantic	REST API, WebSocket, request validation
Front end	HTML5, CSS3, JavaScript	Client and Admin pages (no framework), canvas map, SVG illustrations
Speech (bonus)	Web Speech API	Speech recognition and read-back in the browser
Testing	pytest, unittest, Playwright	Unit, integration and browser end-to-end tests
Tools	Git, GitHub, VS Code	Version control, pull requests, development on Ubuntu 24.04

Table 9: Technologies used.

8 Development Milestones

Milestone	Delivered
Phase 1.1 – Simulation	Restaurant Lounge world, BellaBot model with LiDAR and odometry, Gazebo–ROS bridge
Phase 1.2 – Mapping and navigation	SLAM map, AMCL localization at the dock, Nav2 configuration, 24 table waypoints
Phase 1.3 – Delivery logic	Task queue manager with edit, delete and preemption; auto-docking; Mission Control simulator; 9 unit tests
Phase 2.1 – Dynamic obstacles	Pedestrian model and controller (4 modes), 10 Hz local costmap, RViz markers
Phase 2.2 – Complex layouts	Dar Tunibot world and map, inflation tuning for 1.20 m doorways, multi-stage recovery
Phase 2.3 – Battery	Physical Li-ion model, low / critical / recharge docking policy; 5 integration tests and test report
Phase 3.1 – Web interface	`cstam_web` package: Client and Admin pages, ROS 2 bridge, mock mode; web order delivered in Gazebo
Phase 3.2 – UX and voice	Voice orders (French, English, Darija), arrival estimate, order review, edge cases, automated tests
Phase 3.3 – Design	Nabeul pottery identity, dish illustrations, menu book, table on the floor plan, several orders, Admin control room
Phase 3.4 – Documentation	This report, README and demonstration video

Table 10: Development milestones.

9 Testing Results

9.1 Automated tests

Suite	Phase	Tests	What it checks
`test_system.py`	1	9	Workspace files, 26 waypoints, queue lifecycle (queued → en route → completed), modify and delete, auto-docking idle trigger
`test_system_phase2.py`	2	5	Obstacle detection, doorway clearance, corridor recovery, battery physics, low and critical docking triggers
`cstam_web/test_web.py`	3	16	Order lifecycle follows the task manager, order matching, cancelled / rejected / failed orders, offline detection, battery lock, validation, admin commands, WebSocket
`cstam_web/test_voice.py`	3	28	French, English and Darija sentences, sound-alike spellings, unknown dish reported, table must exist, best alternative kept
`cstam_web/test_e2e_browser.py`	3	7	Voice order delivered end to end, microphone flow, table on the floor plan, menu book, languages, admin map click, several orders
Total		**65**	**All passing**

Table 11: Automated test suites (parametrised cases counted individually).

9.2 Phase 2 test cases

Test case	Input	Result
TC-01 Obstacle detection	Pedestrian 1.0 m ahead, walking at 0.35 m/s	Detected, local costmap marked, robot avoids contact
TC-02 Doorway clearance	1.20 m door, $r_{\text{robot}} = 0.28$, $r_{\text{infl}} = 0.42$, $k = 4.5$	Free corridor 0.64 m, path found through the door
TC-03 Corridor recovery	Nav2 abort in a corridor	Reverse 1.8 s at −0.15 m/s, costmap cleared, order kept
TC-04 Battery physics	Discharge 100 % → 20 %	29.4 V → 21.8 V, 0.29 V sag at 130 W
TC-05 Docking triggers	SoC 24.9 % and 14.9 %	Low and critical dock commands within 15 ms

Table 12: Phase 2 test cases (all passed).

9.3 Scenario tests in Gazebo

Scenario	Observed	Result
Deliveries to all 24 tables (Lounge)	Smooth paths through every aisle, no collision	Pass
Deliveries through 1.20 m arches (Dar Tunibot)	Robot passes on the doorway centreline	Pass
Edit and delete a queued order	Queue updated live, active delivery not disturbed	Pass
Return to dock, then stop docking	Robot drives to the dock; stops and goes IDLE on command	Pass
New order during docking	Docking cancelled, robot turns to the table	Pass
Pedestrian crossing or blocking the aisle	Robot steers around or re-plans, delivery continues	Pass
Battery set to 24 % / 14 %	Home after the queue / emergency return at once	Pass
Order from the web Client page	`TASK-0003` delivered to Table 4; page shows on the way, at your table, delivered	Pass

Table 13: End-to-end scenarios run in Gazebo.

9.4 Issues found and fixed

- **Floating pedestrian.** The first pedestrian model had no gravity and a single link; contact with chairs tipped it 90° and it floated through the room. We rebuilt it on a hidden differential-drive chassis with gravity, a low centre of mass and casters, and drove it with a heading-and-speed controller. It now stays upright and visible to the LiDAR.
- **Robot not going home on critical battery.** The task manager assumed the robot was at the dock because its last pose defaulted to the dock coordinates, so it never sent the Nav2 goal. Using the latest AMCL pose fixed it.
- **Blocked doorways.** Default inflation closed Dar Tunibot's doors; fixed by the tuning in Section 4.4.
- **Voice language.** On an English page, French speech came back as nonsense; the voice panel got its own language choice.

10 Future Improvements

- **Real robot:** run the same stack on a physical base; the web layer needs no change since it only uses ROS 2 topics.
- **People-aware navigation:** predict pedestrian motion (social costmap layers) instead of treating people as obstacles that appear.
- **Fleet:** several robots on one Admin map, with orders assigned to the nearest free robot.
- **Batched deliveries:** serve several tables on one trip using the robot's trays.
- **Kitchen screen:** a page for the kitchen to mark an order ready before the robot leaves.
- **Offline speech:** an on-device speech model (e.g. Whisper) for voice ordering without a cloud service and with better Darija accuracy.
- **Security and analytics:** HTTPS and staff login for the Admin page; delivery times, battery cycles and busiest tables over a day.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[2] S. Macenski, F. Martín, R. White, J. Ginés Clavero, "The Marathon 2: A Navigation System," *IEEE/RSJ IROS*, 2020.

[3] S. Macenski, I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, 2021.

[4] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[5] D. Fox, W. Burgard, S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997.

[6] Open Robotics, "ROS 2 Jazzy documentation," https://docs.ros.org/en/jazzy/.

[7] Open Robotics, "Gazebo Harmonic documentation," https://gazebosim.org/docs/harmonic.

[8] Nav2 project, "Nav2 documentation," https://docs.nav2.org/.

[9] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com/.

[10] W3C Community Group, "Web Speech API," https://webaudio.github.io/web-speech-api/.

[11] Microsoft, "Playwright for Python," https://playwright.dev/python/.

[12] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018.